OxcerAI

Technical White Paper

Local-First Document Intelligence for non-developers

v0.1 · March 2026

github.com/your-repo/oxcer

MIT License

Contents

1 Introduction **2**
1.1 The Problem . . . 2
1.2 What Oxcer Solves . . . 2
1.3 Target Users . . . 2

2 Architecture **2**
2.1 Overview . . . 2
2.2 Request Flow . . . 3
2.3 Plan-First Orchestration . . . 3
2.4 On-Device Inference . . . 3
2.5 Inference Backend Abstraction . . . 3
2.6 FFI Design . . . 4

3 Data & Privacy Model **4**
3.1 What Stays on Device . . . 4
3.2 What May Leave the Machine . . . 5
3.3 DLP Scrubbing . . . 5
3.4 Threat Model . . . 5

4 Local Intelligence Design **5**
4.1 Philosophy . . . 5
4.2 Workflow Memory . . . 6
4.3 Document Relationship Graph . . . 6
4.4 Personalized Command Set . . . 6
4.5 The Architecture of Accumulated Value . . . 7

5 User Experience **7**
5.1 Design Principles . . . 7
5.2 Representative Flow: Document Summarization . . . 7
5.3 Current Limitations . . . 7

6 Roadmap **8**
6.1 v0.1 — Current . . . 8
6.2 v1.0 — Target . . . 8
6.3 Beyond v1.0 . . . 8
6.3.1 Cross-Platform & Hardware Expansion . . . 8

Introduction

The Problem

Knowledge workers process dozens of documents every day — meeting notes, reports, contracts, changelogs. The default answer has been to upload those documents to a cloud service. The model reads them on a remote server. The text travels over a network. It is logged, cached, possibly retained.

For a large class of users — legal, finance, healthcare, independent researchers, anyone working under an NDA — this trade-off is unacceptable. The document cannot leave the machine. The model cannot be trusted unless it runs locally.

The alternative until now has been to run a local model manually: install `llama.cpp`, download a GGUF, write a shell script, pipe a file through it. This works for engineers. It does not work for anyone else.

What Oxcer Solves

Oxcer bridges these two failure modes. It is a macOS application that:

- runs a quantized Llama 3 8B model entirely on-device via Metal,
- accepts natural-language file commands from a chat interface,
- reads, summarizes, and operates on local files without any network call during inference,
- requires no terminal interaction from the end user.

> **Core invariant:** No text from your documents is ever sent to a server. Inference happens on your machine. Nothing leaves.

Target Users

Oxcer v0.1 is built for two audiences simultaneously:

- **Privacy-conscious knowledge workers** — people who need document summarization but cannot use cloud AI.
- **Engineers evaluating local AI architectures** — people who want to understand how a production-grade local agent loop is designed.

Both audiences are served by making the architecture transparent. This document is written for the second group, with the first group's constraints as the design driver.

Architecture

Overview

Oxcer is structured as three distinct layers that communicate across two well-defined boundaries:

1. **oxcer-core** — pure Rust library: agent orchestration, LLM engine abstraction, security policy, semantic routing, plugin system.
2. **oxcer_ffi** — UniFFI bridge layer: exposes oxcer-core to Swift via a generated C ABI. Manages global engine singletons.
3. **OxcerLauncher** — SwiftUI macOS application: chat UI, tool executor, approval gate, settings.

The key architectural decision is that **all agent state lives in Rust**. Swift is stateless with respect to the agent loop. At each step, Swift passes an opaque JSON blob back to Rust unchanged; Rust deserializes, advances the state machine, re-serializes, and returns the next action.

Request Flow

A complete document summarization request flows as follows:

1. User types `"Summarize report.md in Downloads"` in the chat UI.
2. `AppViewModel` builds an `AgentEnvironment` and calls `AgentRunner.run()`.
3. `AgentRunner` calls `ffi_agent_step()` with `session_json = nil` (first step).
4. Rust deserializes the nil session, calls `start_session()`.
5. `semantic_router::route()` classifies the input via keyword heuristics — no LLM involved.
6. `planning::start_session()` builds the plan: `[FsReadFile("report.md"), LlmGenerate]`.
7. Rust returns `FfiStepOutcome{status: "need_tool", intent: fs_read_file}`.
8. Swift executes the file read via `FileManager`, returns contents.
9. Rust receives the file contents, substitutes `{{FILE_CONTENTS}}` into the prompt, advances to `LlmGenerate`.
10. Swift calls `generate_text(prompt)` via FFI.
11. Rust runs inference on-device via `llama-cpp-2` + Metal.
12. Result returns to Swift; `AgentRunner` receives `status: "complete"`.
13. `AppViewModel` appends the summary as a chat message.

Plan-First Orchestration

The orchestrator follows a **plan-first** model rather than a ReAct loop. Before executing any tool, `start_session()` builds the complete `Vec<ToolCallIntent>` upfront. The execution loop in `next_action()` then advances through the plan mechanically.

The model is never asked "what should I do next?" The planner decides deterministically. This makes the system:

- **Auditable** — the full plan is logged at session start.
- **Testable** — plan construction is pure functions with no side effects.
- **Predictable** — users and developers can reason about what the agent will do before it does it.

The trade-off is reduced dynamic flexibility. This is acceptable at v0.1 and addressed by the `ExpansionKind` mechanism for future workflows.

On-Device Inference

The local model is **Meta-Llama-3-8B-Instruct** at Q4_K_M quantization (4.9 GiB). It is loaded via `llama-cpp-2`, the Rust binding to `llama.cpp`, with `n_gpu_layers=99` — full Metal offload to the Apple Silicon GPU.

Key inference decisions:

- **Fresh context per call** — a new `LlamaContext` is created for each `generate_text` invocation. No shared KV cache across requests. Simpler memory management; re-prefill cost is acceptable at current document sizes.
- **Context ceiling** — `CONTEXT_LIMIT = 8192` tokens. File content is clamped to `FS_RESULT_MAX_CHARS = 4000` characters before injection into the prompt, leaving headroom for the system prompt, prompt frame, and generation.
- **Blocking inference** — `generate_text` runs inside `tokio::spawn_blocking` to avoid starving the async executor.

Inference Backend Abstraction

The choice of llama.cpp + GGUF + Metal is a **v0.1 decision, not a permanent constraint**. The entire inference stack sits behind the `LlmEngine` trait in `oxcer-core`:

```
pub trait LlmEngine: Send + Sync {
    fn generate(&self, prompt: &str, params: &GenerationParams)
        -> Result<Option<String>, LlmError>;
}
```

`LocalPhi3Engine`, `CloudLlmEngine`, and any future backend implement this single interface. The orchestrator, FFI layer, and Swift UI are entirely unaware of which engine is active.

This abstraction was designed with cross-platform expansion in mind. `llama.cpp` is the right choice for Apple Silicon today — Metal offload, mature GGUF ecosystem, zero dependencies. But as Oxcer expands to Windows and Linux, the GGUF format and llama.cpp runtime become a bottleneck: GGUF has no native CUDA graph support, no ROCm path, and no NPU acceleration story.

The planned migration path is to **ONNX Runtime** as the cross-platform inference backend, behind the same `LlmEngine` trait. ONNX Runtime provides a unified execution provider model that covers:

- **CUDA** — NVIDIA GPU acceleration on Windows and Linux
- **ROCm** — AMD GPU support
- **DirectML** — Windows-native GPU inference
- **QNN** — Qualcomm Hexagon NPU (Snapdragon X Elite)
- **OpenVINO** — Intel NPU and integrated GPU

Because the swap happens entirely within `oxcer-core`, no changes are required in the FFI layer or the Swift UI. The `LlmEngine` boundary is the migration seam.

FFI Design

The boundary between Rust and Swift is managed by UniFFI 0.28, which generates the Swift wrapper and C bridging header from Rust type signatures.

- No manual `unsafe` from Swift.
- No `ffi_string_free` memory management.
- ABI checksum verification at startup — if Swift was compiled against stale bindings, the app traps with a clear `apiChecksumMismatch` error rather than silently producing corrupt output.

Binding freshness is enforced by three independent mechanisms: unit tests, a CI job that regenerates bindings from the release dylib and diffs them, and a local pre-push hook.

Data & Privacy Model

What Stays on Device

Data	Location
Document contents	Never leave the machine. Read by Swift `FileManager`, passed to Rust in-process, passed to local LLM in-process.
Model weights	`{appConfigDir}/models/llama3/model.gguf`. Downloaded once from HuggingFace. Never re-uploaded.
Session state	In-memory JSON blob. Not persisted across launches in v0.1.
API keys (cloud)	Stored in macOS Keychain via `KeychainHelper.swift`. Never logged.
Structured logs	Written to stdout via `tracing` subscriber. Never sent to a remote endpoint.

What May Leave the Machine

- **Model download** — the initial GGUF download is an outbound HTTP GET to HuggingFace. No document content is involved.
- **Cloud engine (opt-in)** — if the user explicitly configures a cloud provider key in Settings, `generate_text` routes to that provider instead of the local model. The user opts into this explicitly. It is off by default.

DLP Scrubbing

Before any text is embedded into an `LlmGenerate` task string, `prompt_sanitizer::sanitize_task_for_llm()` runs a regex-based scan to redact credentials, API keys, JWTs, and PEM keys. Since local inference makes no network calls, the primary purpose is to prevent sensitive strings from appearing in structured log output.

Threat Model

Threat	Mitigation	Status
Documents exfiltrated to cloud	Local-only inference by default	Mitigated
Credentials leaked to LLM	DLP scrubber on all prompts	Mitigated
Silent filesystem mutation	Approval gate on all write ops	Mitigated
LLM narrating tool calls to user	Narration sanitizer on task sessions	Mitigated
Stale FFI bindings causing memory corruption	ABI checksum + CI freshness check	Mitigated
Unsigned binary blocked by Gatekeeper	Notarization (v1.0)	Planned

Local Intelligence Design

Philosophy

Most local AI tools treat the model as stateless. Every session starts from zero. The user's context, habits, and document history are discarded. The tool is always a stranger.

Oxcer is designed around the opposite principle: **intelligence accumulates with use**. Not in the cloud. Not in a vendor's training pipeline. On the user's machine, in the user's filesystem, attached to the user's documents.

This has a specific structural consequence worth stating explicitly.

On day one, Oxcer summarizes a file. On day thirty, it knows which files you touch every Monday, which documents belong to the same project, and what "weekly report" means in your vocabulary. On month six, the cost of switching to another tool is no longer just the inconvenience of reinstalling software — it is the loss of a context graph that took six months to build and exists nowhere else.

This is not lock-in by contract or data export restriction. It is **lock-in by accumulated value** — the kind that emerges when a tool becomes genuinely personal. The workflow pattern *"meeting notes → summarize → move to team folder"* stored in Oxcer is not a file you can copy. It is a learned representation of how one person works. That representation lives locally because that is the only place it is true.

Three concrete mechanisms implement this philosophy. Their underlying code exists in the v0.1 codebase, fully tested and without stubs. Connecting them to the main agent loop is the primary engineering work of v1.0.

Workflow Memory

What it is: A persistent, queryable log of actions the agent has taken on behalf of the user. Not a conversation history — a fact store.

Implementation status: `memory.rs` is fully implemented with no stubs. Key capabilities:

- `Memory::append_fact()` — derives tags automatically (file, dir, search, summary) from the content via keyword inference.
- `Memory::query(keyword)` — keyword scoring, returns top-10 relevant facts.
- `Memory::as_context()` — wraps query results in a `[MEMORY_CONTEXT]` block for injection into prompts.
- `Memory::compact()` — automatically drops oldest facts beyond 100, inserts a `[compacted]` marker.
- Persistence format: human-readable Markdown on disk.

v1.0 integration: `ffi_agent_step` will call `Memory::append_fact()` after each completed session and `Memory::as_context()` at session start to inject relevant history into the planner context.

Why it matters: A user who summarizes the same project folder every Monday does not need to describe it each time. The agent remembers. Over time, this memory becomes an asset that reflects one person's workflow — and that asset lives exclusively on their machine.

Document Relationship Graph

What it is: A SQLite-backed episodic memory that records which documents were processed together, what queries were run, and what observations were produced. Over time this builds a graph of document relationships that only makes sense in the context of one user's filesystem.

Implementation status: `db.rs` is fully implemented. The schema is a single `episodic_memory` table with WAL mode enabled. Key operations:

- `insert_fact(query, observation)` — records a query/result pair.
- `get_recent_context(limit)` — retrieves the n most recent facts by insertion order.
- `fact_count()` — used for compaction decisions.

v1.0 integration: The episodic store will be extended to record document co-occurrence. When documents A and B are processed in the same session, a relationship edge is written. The graph grows denser with each session. A user who has processed an entire project's documents through Oxcer carries a relationship map that no other tool has — and that map cannot be exported, because it is not a list of files. It is a history of how those files were used together.

Why it matters: The longer the graph grows, the more Oxcer understands the structure of a user's work. Moving to a different tool means starting that graph from zero.

Personalized Command Set

What it is: A layer that detects recurring command patterns and allows users to define short aliases that expand to full workflows.

Implementation status: `agent_session_log.rs` captures the full structured record needed to detect patterns — router decision, tool trace sequence, model used, final answer — with DLP scrubbing applied before any write. The log builder is complete; the pattern-detection layer is the v1.0 work.

v1.0 integration: When the system detects that a user runs the same workflow more than n times, it surfaces a suggestion: "Save this as a shortcut?" The shortcut is stored locally and resolves to the full `SessionState` plan at invocation time.

Why it matters: `"oxcer weekly-report"` as a first-class command, defined by the user from their own usage pattern, is not a feature. It is a vocabulary that belongs to one person. That vocabulary is the clearest expression of why local accumulation matters: the tool has learned to speak the user's language, and that language exists nowhere else.

The Architecture of Accumulated Value

Cloud AI tools accumulate user context on the vendor's infrastructure. The user's value is stored where the vendor can see it, aggregate it, and use it to improve a shared model.

Oxcer's local intelligence layers invert this. Workflow patterns, document relationships, and command vocabulary are assets that live on the user's machine. They cannot be aggregated. They cannot be retrained on. They belong to the user in a literal, filesystem sense.

This is not a privacy feature layered on top of a cloud architecture. It is a different architecture entirely. The local accumulation of intelligence is the product.

User Experience

Design Principles

1. **Terminal is an implementation detail.** The user never opens a terminal. `llama.cpp` runs inside the app. Model download happens through a progress overlay. The shell exists internally as a tool the agent can call; it is not a user-facing surface.
2. **Explicit over implicit.** The agent does not silently mutate the filesystem. Every write, delete, rename, or move surfaces an approval bubble before execution. Read-only operations proceed without interruption.
3. **Honest about limitations.** When the model narrates its own tool calls instead of summarizing, the narration sanitizer surfaces a clear error rather than showing internal monologue as output.

Representative Flow: Document Summarization

1. User launches OxcerLauncher. If no model is present, a download overlay shows progress for the 4.9 GiB GGUF.
2. User types: `"Summarize report.md in Downloads"`
3. The agent reads the file silently (no approval required — read-only).
4. A loading indicator is shown while inference runs.
5. The summary appears as an assistant message in the chat thread.

No configuration required. No file picker. No model selection. The user names the file; the agent finds it and summarizes it.

Current Limitations

The following limitations are present in v0.1 and are being addressed:

- **No streaming output.** The full summary is buffered before display. Users wait silently for 15–60 seconds on M1. Streaming via incremental token callbacks is the first v1.0 UX fix.
- **File truncation.** Documents longer than 4,000 characters are truncated with a notice appended to the output.
- **English directory names only.** The semantic router matches "Downloads", "Desktop", "Documents" as exact substrings. Non-English macOS installations are not yet supported.
- **PDF not supported.** v0.1 reads `.md`, `.txt`, and other UTF-8 plain-text formats. PDF requires a text extraction layer planned for v1.0.

Roadmap

v0.1 — Current

- Single-file summarization (`.md`, `.txt` and other UTF-8 plain-text formats)
- Local Llama 3 8B Q4_K_M inference via Metal
- Human-in-the-loop approval for all write operations
- DLP scrubbing on all prompt text
- Cloud provider opt-in (OpenAI, Anthropic, Gemini, Grok)
- Narration sanitizer
- FFI binding freshness enforcement (CI + pre-push hook)
- Fully implemented `memory.rs`, `db.rs`, `agent_session_log.rs` — awaiting v1.0 wiring

v1.0 — Target

- **Streaming output** — incremental token display, no silent wait
- **Multi-file summarization** — Workflow 2 plan expansion, currently disabled
- **Workflow Memory wired** — `memory.rs` connected to `ffi_agent_step`; session history injected at plan time
- **Document Relationship Graph** — episodic store records document co-occurrence across sessions
- **Personalized Command Set** — pattern detection on session logs; user-defined workflow shortcuts
- **PDF support** — text extraction layer for `.pdf` files
- **App notarization** — Developer ID signing; Gatekeeper-safe distribution
- **Context window expansion** — larger model or sliding-window chunking for documents beyond 4,000 characters

Beyond v1.0

- Folder-level operations (Workflow 3: move-to-dir)
- Cross-session document graph queries
- Non-English directory name support

Cross-Platform & Hardware Expansion

Oxcer is designed to run on every major platform and accelerator. The `LlmEngine` trait abstraction means the inference backend can be swapped without touching the orchestrator, FFI layer, or UI.

The planned path:

- **Windows & Linux** — the Rust core is platform-agnostic. Native launchers are planned.
- **GGUF → ONNX Runtime migration** — replace `llama-cpp-2` with an ONNX Runtime backend behind the same `LlmEngine` interface. This unlocks hardware acceleration across the full device landscape:
  - CUDA (NVIDIA)
  - ROCm (AMD)
  - DirectML (Windows GPU)
  - QNN / Hexagon NPU (Qualcomm Snapdragon X)
  - OpenVINO (Intel NPU + iGPU)
- **Metal retained on Apple Silicon** — the current llama.cpp + Metal path remains for macOS. ONNX Runtime's CoreML execution provider is an alternative migration target if it reaches parity.

The goal: a single codebase that runs local inference on any consumer device — Apple Silicon, Snapdragon, NVIDIA, AMD — without cloud dependency on any of them.

Appendix

Model & License

- **Model:** Meta-Llama-3-8B-Instruct-Q4_K_M.gguf
- **Model license:** Meta Llama 3 Community License (consent gate shown at first launch)
- **Oxcer license:** MIT
- **Inference backend:** llama-cpp-2 (Rust bindings to llama.cpp)
- **FFI layer:** UniFFI 0.28 (Mozilla)

System Requirements

- Apple Silicon Mac (M1 or later)
- macOS 13 Ventura or later
- 8 GB RAM minimum; 16 GB recommended when running alongside other apps
- 6 GB free disk space for model download

Repository

- Source: `https://github.com/oxcer/oxcer-agent`
- Contributing guide: `CONTRIBUTING.md`
- Security policy: `docs/security.md`
- Architecture notes: `docs/ARCHITECTURE.md`